



AURORA ROBOTICS

Mobile Robot Kinematics

A Guided Textbook Chapter

Based on the AuroraMR Library

Models Covered

- Two-Wheel / Unicycle
- Ackermann / Bicycle
- Differential Drive
- Mecanum (Holonomic)

By:

Bakel Bakel Begedum · CTO & Chief Instructor, Aurora Robotics

Tikuochi Iheukwumere · Intern & Instructor, Aurora Robotics

For each model:

Derivation from first principles · Complete equation set · Step-by-step solutions ·
Numerical worked examples · Python (AuroraMR) snippets

April 2026

Contents

1	Foundations: Pose, Frames, and Conventions	3
1.1	The Robot's Pose	3
1.1.1	World Frame and Axis Convention	3
1.1.2	Body Frame	4
1.1.3	Angle Wrapping	4
1.2	The Rotation Matrix	5
2	Two-Wheel / Unicycle Kinematics	7
2.1	Model Description	7
2.2	Continuous Kinematic Equations	7
2.3	Euler Integration (Discrete Time)	7
2.4	Worked Numerical Example	8
2.5	AuroraMR Code	9
2.6	Velocity Limits	9
3	Differential Drive Kinematics	10
3.1	Model Description	10
3.2	From Wheel Speeds to Twist	10
3.3	Special Cases	11
3.4	In-Place Rotation	11
3.5	Worked Numerical Examples	11
3.6	AuroraMR Code	12
4	Ackermann / Bicycle Kinematics	14
4.1	Model Description	14
4.2	Bicycle Model Equations	14
4.3	Ideal Ackermann Steering Geometry	15
4.4	Worked Numerical Examples	16
4.5	AuroraMR Code	17
5	Mecanum (Holonomic) Kinematics	18
5.1	Model Description	18
5.2	Inverse Kinematics: Wheel Speeds to Body Twist	18
5.3	World Frame Integration	19
5.4	Inverse Kinematics: Desired Twist $\rightarrow$ Wheel Speeds	20
5.5	Special Motion Patterns	20
5.6	Worked Numerical Examples	20
5.7	AuroraMR Code	22

6	Wheel Contact Point Geometry	23
6.1	Why Track Contact Points?	23
6.2	Two-Wheel / Differential Contacts	23
6.3	Four-Wheel (Ackermann) Contacts	24
6.4	Mecanum Four-Wheel Contacts	24
7	Numerical Accuracy and Integration Error	25
7.1	Euler Method Error Analysis	25
7.2	Runge-Kutta 4 (RK4) Alternative	25
8	Summary of All Equations	27
9	Practice Problems	29
A	Quick Reference: AuroraMR Parameter Defaults	30

Chapter 1

Foundations: Pose, Frames, and Conventions

1.1 The Robot's Pose

A mobile robot moving on a flat surface has three degrees of freedom. Its complete *configuration* — or **pose** — is described by three numbers:

Definition: Pose

The pose of a planar robot is the triple

$$\mathbf{q} = (x, y, \theta) \in \mathbb{R}^2 \times (-\pi, \pi]$$

where (x, y) is the position of the robot's reference point in the *world frame*, and θ is the *heading* (orientation) of the robot's forward axis.

1.1.1 World Frame and Axis Convention

We adopt the following convention throughout, which matches the AuroraMR library:

AuroraMR Coordinate Convention

- x -axis points **right** (East).
- y -axis points **up** (North).
- $\theta = 0$: robot faces **North** (positive y).
- θ increases **counter-clockwise** (CCW) when viewed from above.
- $\theta = \frac{\pi}{2}$: robot faces **West**.
- $\theta = \pi$ (or $-\pi$): robot faces **South**.
- $\theta = -\frac{\pi}{2}$ (or $\frac{3\pi}{2}$): robot faces **East**.

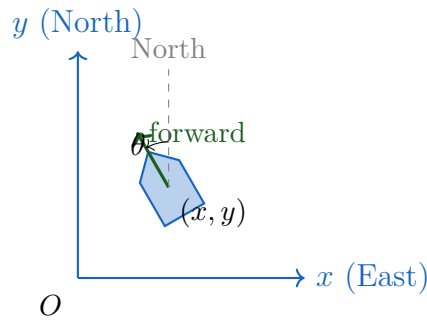


Figure 1.1: World frame: x right, y up. Heading θ is CCW from North.

1.1.2 Body Frame

The **body frame** is fixed to the robot:

- Body $+y_b$ axis: forward direction of the robot.
- Body $+x_b$ axis: right side of the robot.

1.1.3 Angle Wrapping

Because heading is periodic with period 2π , we always keep θ in the interval $(-\pi, \pi]$ using:

Step-by-Step Derivation

The idea is to shift θ by $+\pi$ (so the desired range $(-\pi, \pi]$ maps to $(0, 2\pi]$), apply modulo 2π (always positive in Python for positive divisor), then shift back by $-\pi$.

Worked Example: wrap_pi numerical

Input 1: $\theta = 200^\circ = 3.491$ rad.

Step 1. Add π : $3.491 + 3.1416 = 6.632$

Step 2. Modulo 2π : $6.632 \bmod 6.2832 = 0.349$

Step 3. Subtract π : $0.349 - 3.1416 = -2.793$ rad $= -160^\circ$

200° and -160° represent the same heading. ✓

Input 2: $\theta = -400^\circ = -6.981$ rad.

Step 1. $-6.981 + 3.1416 = -3.839$

Step 2. $-3.839 \bmod 6.2832 = 2.444$ (Python: $-3.839 + 6.2832$)

Step 3. $2.444 - 3.1416 = -0.698$ rad $= -40^\circ$ ✓

1.2 The Rotation Matrix

To transform a vector from the body frame to the world frame, we rotate it by θ :

Key Equation: 2-D Rotation Matrix

$$\mathbf{R}(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$$

Given a point $\mathbf{p}_b = (p_x, p_y)^\top$ in the body frame, its world-frame coordinates are:

$$\mathbf{p}_w = \mathbf{R}(\theta) \mathbf{p}_b + \begin{pmatrix} x \\ y \end{pmatrix}$$

The Forward Direction

The body $+y_b = (0, 1)^\top$ axis (forward) in the world frame is:

$$\mathbf{R}(\theta) \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} -\sin \theta \\ \cos \theta \end{pmatrix}$$

This vector $\mathbf{f} = (-\sin \theta, \cos \theta)^\top$ appears in every integration equation.

Worked Example: Rotation Matrix

Robot at heading $\theta = 45^\circ = \pi/4$.

$$\mathbf{R}(45^\circ) = \begin{pmatrix} \cos 45^\circ & -\sin 45^\circ \\ \sin 45^\circ & \cos 45^\circ \end{pmatrix} = \begin{pmatrix} 0.707 & -0.707 \\ 0.707 & 0.707 \end{pmatrix}$$

Body point at left wheel $(p_x, p_y) = (-0.2, 0)$ (0.2 m left of centre):

$$\mathbf{p}_w = \begin{pmatrix} 0.707 & -0.707 \\ 0.707 & 0.707 \end{pmatrix} \begin{pmatrix} -0.2 \\ 0 \end{pmatrix} + \begin{pmatrix} 1.0 \\ 2.0 \end{pmatrix} = \begin{pmatrix} -0.141 \\ -0.141 \end{pmatrix} + \begin{pmatrix} 1.0 \\ 2.0 \end{pmatrix} = \begin{pmatrix} 0.859 \\ 1.859 \end{pmatrix}$$

Left wheel is at world coordinates (0.859, 1.859).

Chapter 2

Two-Wheel / Unicycle Kinematics

2.1 Model Description

The **unicycle model** (also called the two-wheel model) is the simplest useful kinematic model. The robot is treated as a point controlled by:

- v — forward (linear) speed at the robot centre [m/s]
- ω — yaw rate (angular velocity) [rad/s], positive = CCW

Assumption

The unicycle model assumes **no lateral slip**. The robot can only move in the direction it is facing.

2.2 Continuous Kinematic Equations

In continuous time, the pose evolves as:

$$\frac{dx}{dt} = -v \sin \theta, \quad \frac{dy}{dt} = v \cos \theta, \quad \frac{d\theta}{dt} = \omega$$

These are the *nonholonomic constraint equations* for a unicycle. The minus sign on $\dot{x}$ (and plus on $\dot{y}$) arises directly from the North-is-zero convention: the forward direction is $(-\sin \theta, \cos \theta)$.

2.3 Euler Integration (Discrete Time)

In simulation we advance the pose by a small time step Δt . Using the first-order Euler method:

Key Equation: Unicycle Euler Integration

$$\begin{aligned}
x(t + \Delta t) &= x(t) - v \sin(\theta(t)) \Delta t \\
y(t + \Delta t) &= y(t) + v \cos(\theta(t)) \Delta t \\
\theta(t + \Delta t) &= \text{wrap}_\pi(\theta(t) + \omega \Delta t)
\end{aligned}$$

Full Derivation of the x and y Updates

1. The displacement vector in time Δt is: $\Delta \mathbf{p} = v \Delta t \cdot \mathbf{f}(\theta)$ where $\mathbf{f}(\theta) = (-\sin \theta, \cos \theta)^\top$ is the forward unit vector.
2. Therefore:

$$\Delta x = v \Delta t \cdot (-\sin \theta) = -v \sin \theta \Delta t$$

$$\Delta y = v \Delta t \cdot \cos \theta = v \cos \theta \Delta t$$

3. The heading change over Δt is simply:

$$\Delta \theta = \omega \Delta t$$

4. Apply wrap_π to keep $\theta \in (-\pi, \pi]$.

2.4 Worked Numerical Example**Worked Example: Unicycle Integration — Three Steps**

Setup: Robot starts at $\mathbf{q}_0 = (0, 0, 0)$ (facing North). Commands: $v = 1.0$ m/s, $\omega = 0.5$ rad/s, $\Delta t = 0.1$ s.

Step 1: $t = 0 \rightarrow 0.1$ s.

$$x_1 = 0 - (1.0) \sin(0) \cdot (0.1) = 0 - 0 = 0.000 \text{ m}$$

$$y_1 = 0 + (1.0) \cos(0) \cdot (0.1) = 0 + 0.100 = 0.100 \text{ m}$$

$$\theta_1 = \text{wrap}_\pi(0 + 0.5 \times 0.1) = \text{wrap}_\pi(0.050) = 0.050 \text{ rad}$$

Step 2: $t = 0.1 \rightarrow 0.2$ s, using $\theta = 0.050$ rad.

$$x_2 = 0 - (1.0) \sin(0.050) \cdot (0.1) = 0 - 0.00500 = -0.005 \text{ m}$$

$$y_2 = 0.100 + (1.0) \cos(0.050) \cdot (0.1) = 0.100 + 0.09988 = 0.200 \text{ m}$$

$$\theta_2 = \text{wrap}_\pi(0.050 + 0.050) = 0.100 \text{ rad}$$

Step 3: $t = 0.2 \rightarrow 0.3$ s, using $\theta = 0.100$ rad.

$$x_3 = -0.005 - (1.0) \sin(0.100) \cdot (0.1) = -0.005 - 0.00998 = -0.015 \text{ m}$$

$$y_3 = 0.200 + (1.0) \cos(0.100) \cdot (0.1) = 0.200 + 0.09950 = 0.300 \text{ m}$$

$$\theta_3 = \text{wrap}_\pi(0.100 + 0.050) = 0.150 \text{ rad} \approx 8.59^\circ$$

Step	x (m)	y (m)	θ (rad)
0	0.000	0.000	0.000
1	0.000	0.100	0.050
2	-0.005	0.200	0.100
3	-0.015	0.300	0.150

The robot curves left (CCW) as it moves north, consistent with positive ω .

2.5 AuroraMR Code

Listing 2.1: Unicycle simulation in AuroraMR

```

1 import amr, math
2
3 s = amr.MotionSession.create(
4     amr.pose(0, 0, 0),          # x=0, y=0, theta=0 (facing
5     amr.KinematicsModel.TWO_WHEEL,
6     dt=0.02,
7     unicycle=amr.UnicycleParams(track_width=0.4,
8     max_linear_speed=1.5)
9 )
10 s.forward(1.0, speed=0.8)      # move 1 m forward at 0.8 m/s
11 s.turn_left(math.pi/2, 1.0)   # turn 90 deg CCW at 1.0 rad/s
12 s.forward(0.5, speed=0.8)     # move 0.5 m forward
13
14 amr.plot_motion(s)             # static plot
15 amr.play_motion(s, interval_ms=25) # live animation

```

2.6 Velocity Limits

The AuroraMR library clamps commands before integration:

$$v_{\text{clamped}} = \max(-v_{\text{max}}, \min(v_{\text{max}}, v)), \quad \omega_{\text{clamped}} = \max(-\omega_{\text{max}}, \min(\omega_{\text{max}}, \omega))$$

This is equivalent to the element-wise saturation function $\text{sat}(u, u_{\text{max}}) = \text{sign}(u) \cdot \min(|u|, u_{\text{max}})$.

Chapter 3

Differential Drive Kinematics

3.1 Model Description

A **differential drive** robot has two independently driven wheels mounted on the same axle. Let:

- v_L — linear speed of the *left* wheel at ground contact [m/s]
- v_R — linear speed of the *right* wheel at ground contact [m/s]
- W — track width (distance between wheel contact points) [m]

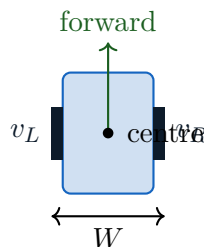


Figure 3.1: Differential drive: two independently actuated wheels, track width W .

3.2 From Wheel Speeds to Twist

The **no-slip rolling constraint** on each wheel gives:

- Right wheel contact at $+W/2$ from centre: $v_R = v + \omega \cdot W/2$
- Left wheel contact at $-W/2$ from centre: $v_L = v - \omega \cdot W/2$

Solving these two equations simultaneously for v and ω :

Key Equation: Differential Drive — Twist from Wheel Speeds

$$v = \frac{v_R + v_L}{2},$$

$$\omega = \frac{v_R - v_L}{W}$$

Full Derivation

$$v_L = v - \omega \frac{W}{2} \quad (\text{left wheel})$$

$$v_R = v + \omega \frac{W}{2} \quad (\text{right wheel})$$

Add (left wheel) and (right wheel):

$$v_L + v_R = 2v \implies v = \frac{v_L + v_R}{2}$$

Subtract (left wheel) from (right wheel):

$$v_R - v_L = \omega W \implies \omega = \frac{v_R - v_L}{W}$$

Once (v, ω) are known, we use the unicycle integrator (chapter 2 equations) directly.

3.3 Special Cases

Condition	Result	Geometry
$v_L = v_R$	$v = v_R, \omega = 0$	Straight line
$v_L = -v_R$	$v = 0, \omega = 2v_R/W$	Rotate in place
$v_L = 0$	$v = v_R/2, \omega = v_R/W$	Arc, left wheel as pivot
$v_R = 0$	$v = v_L/2, \omega = -v_L/W$	Arc, right wheel as pivot

3.4 In-Place Rotation

For a desired yaw rate ω , set:

$$v_L = -\omega \frac{W}{2}, \quad v_R = +\omega \frac{W}{2}$$

Verification: $v = (v_R + v_L)/2 = 0 \checkmark$ $\omega_{\text{check}} = (v_R - v_L)/W = \omega W/W = \omega \checkmark$

3.5 Worked Numerical Examples

Worked Example: Differential Drive — Arc Turn

Setup: Track width $W = 0.40$ m. Commands: $v_L = 0.3$ m/s, $v_R = 0.7$ m/s, $\Delta t = 0.05$ s. Initial pose: $\mathbf{q}_0 = (1.0, 2.0, 0)$ (North).

Step 1: Compute twist.

$$v = \frac{0.7 + 0.3}{2} = 0.500 \text{ m/s}, \quad \omega = \frac{0.7 - 0.3}{0.40} = \frac{0.4}{0.4} = 1.000 \text{ rad/s (CCW)}$$

Step 2: Unicycle integrate.

$$\begin{aligned}x_1 &= 1.0 - (0.5) \sin(0) \cdot (0.05) = 1.0 - 0 = 1.000 \text{ m} \\y_1 &= 2.0 + (0.5) \cos(0) \cdot (0.05) = 2.0 + 0.025 = 2.025 \text{ m} \\ \theta_1 &= \text{wrap}_\pi(0 + 1.0 \times 0.05) = 0.050 \text{ rad}\end{aligned}$$

Turning radius check: $R = v/\omega = 0.5/1.0 = 0.5 \text{ m}$. This is the radius from the robot centre to the ICC (Instantaneous Centre of Curvature).

Worked Example: In-Place Rotation

Setup: $W = 0.40 \text{ m}$. Desired $\omega = 1.5 \text{ rad/s}$ CCW. $\Delta t = 0.02 \text{ s}$. Initial: $\theta = 0$.

Step 1: Compute wheel speeds.

$$v_L = -1.5 \times \frac{0.40}{2} = -0.30 \text{ m/s}, \quad v_R = +1.5 \times \frac{0.40}{2} = +0.30 \text{ m/s}$$

Step 2: Compute twist.

$$v = \frac{0.30 + (-0.30)}{2} = 0 \text{ m/s}, \quad \omega = \frac{0.30 - (-0.30)}{0.40} = \frac{0.60}{0.40} = 1.5 \text{ rad/s} \checkmark$$

Step 3: Integrate.

$$x_1 = x_0, \quad y_1 = y_0, \quad \theta_1 = \text{wrap}_\pi(0 + 1.5 \times 0.02) = 0.030 \text{ rad}$$

The robot rotates in place. After 209 steps: $\theta = 209 \times 0.030 \approx 6.27 \approx 2\pi$ (one full revolution).

3.6 AuroraMR Code

Listing 3.1: Differential drive in AuroraMR

```

1  s = amr.MotionSession.create(
2      amr.pose(0, 0, 0),
3      amr.KinematicsModel.DIFFERENTIAL,
4      dt=0.015,
5      differential=amr.DifferentialParams(
6          track_width=0.40,
7          wheel_radius=0.10,      # stored; inputs must be linear m
                                   /s
8          max_wheel_speed=2.0
9      )
10 )
11
12 # High-level commands
13 s.forward(1.0, wheel_speed=0.8)      # straight, equal
    wheel speeds
14 s.turn_right(math.pi/3, angular_speed=1.0)  # CW 60 deg
15

```

```
16 # Low-level: direct wheel speeds for 1.2 s  
17 s.differential_drive_wheels(0.3, 0.8, duration=1.2)
```

Chapter 4

Ackermann / Bicycle Kinematics

4.1 Model Description

The **Ackermann** model describes a four-wheeled vehicle where the *rear wheels are fixed* (no steering) and the *front wheels steer*. The kinematic model is called the *bicycle model* because for analysis we collapse each axle into a single central wheel.

Parameters:

- L — wheelbase (distance front axle to rear axle) [m]
- W — track width (left to right wheel) [m]
- v — speed at the rear axle midpoint [m/s]
- δ — equivalent front steering angle (positive = left turn) [rad]

The robot's pose (x, y, θ) refers to the **rear axle midpoint**.

4.2 Bicycle Model Equations

The angular velocity of the robot for a given steering angle δ is:

Key Equation: Bicycle / Ackermann Yaw Rate

$$\omega = \frac{v \tan \delta}{L}$$

Derivation from Turning Radius

For the front wheel steering at angle δ :

1. The front wheel's velocity must be perpendicular to its axle. This means the robot turns about an ICC located on the rear axle extended.
2. The turning radius (from ICC to rear axle centre) is:

$$R = \frac{L}{\tan \delta}$$

3. Yaw rate is $\omega = v/R$, therefore:

$$\omega = \frac{v}{L/\tan \delta} = \frac{v \tan \delta}{L}$$

The pose integration then uses the unicycle equations with this computed ω :

$$\begin{aligned} x' &= x - v \sin(\theta) \Delta t \\ y' &= y + v \cos(\theta) \Delta t \\ \theta' &= \text{wrap}_{\pi}(\theta + \omega \Delta t) \end{aligned}$$

4.3 Ideal Ackermann Steering Geometry

A key feature of AuroraMR is that it implements *exact* per-wheel steering angles. In a real vehicle, the inner wheel (tighter turn radius) must steer more sharply than the outer wheel so that all four wheels roll without slip.

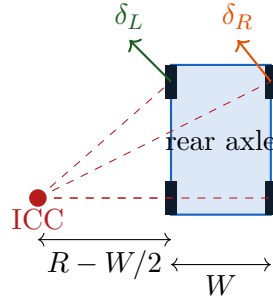


Figure 4.1: Ackermann geometry: both front wheels point toward the same ICC. The inner wheel steers more ($\delta_L > \delta_R$ for a left turn).

Key Equation: Ackermann Per-Wheel Steer Angles

Let $R = L/\tan \delta_c$ (turning radius from rear axle to ICC).

$$\begin{aligned} \delta_{\text{inner}} &= \arctan\left(\frac{L}{R - W/2}\right) \\ \delta_{\text{outer}} &= \arctan\left(\frac{L}{R + W/2}\right) \end{aligned}$$

For a *left* turn ($\delta_c > 0$): inner = left, outer = right. For a *right* turn ($\delta_c < 0$): inner = right, outer = left.

Derivation

For the inner wheel (at $R - W/2$ from the ICC):

$$\tan \delta_{\text{inner}} = \frac{L}{R - W/2} \implies \delta_{\text{inner}} = \arctan\left(\frac{L}{R - W/2}\right)$$

For the outer wheel (at $R + W/2$ from the ICC):

$$\delta_{\text{outer}} = \arctan\left(\frac{L}{R + W/2}\right)$$

Since $R - W/2 < R + W/2$, we have $\delta_{\text{inner}} > \delta_{\text{outer}}$: the inner wheel always steers more sharply.

4.4 Worked Numerical Examples

Worked Example: Bicycle Integration — Gentle Curve

Setup: $L = 0.55$ m, $\delta = 0.20$ rad ($\approx 11.5^\circ$), $v = 0.60$ m/s, $\Delta t = 0.02$ s. Initial pose: $(0, 0, 0)$.

Step 1: Yaw rate.

$$\omega = \frac{(0.60) \tan(0.20)}{0.55} = \frac{0.60 \times 0.2027}{0.55} = \frac{0.1216}{0.55} = 0.2211 \text{ rad/s}$$

Step 2: Integrate at $\theta = 0$.

$$\begin{aligned} x_1 &= 0 - (0.60) \sin(0)(0.02) = 0.000 \text{ m} \\ y_1 &= 0 + (0.60) \cos(0)(0.02) = 0.012 \text{ m} \\ \theta_1 &= \text{wrap}_\pi(0 + 0.2211 \times 0.02) = 0.00442 \text{ rad} \end{aligned}$$

Step 3: Integrate at $\theta_1 = 0.00442$ rad.

$$\begin{aligned} x_2 &= 0 - (0.60) \sin(0.00442)(0.02) = 0 - 0.0000531 \approx -0.0001 \text{ m} \\ y_2 &= 0.012 + (0.60) \cos(0.00442)(0.02) = 0.012 + 0.01200 = 0.024 \text{ m} \\ \theta_2 &= \text{wrap}_\pi(0.00442 + 0.00442) = 0.00884 \text{ rad} \end{aligned}$$

The robot traces a circular arc curving left (CCW).

Worked Example: Ackermann Per-Wheel Steer Angles

Setup: $L = 0.55$ m, $W = 0.36$ m, $\delta_c = 0.40$ rad (22.9°).

Step 1: Turning radius.

$$R = \frac{L}{\tan \delta_c} = \frac{0.55}{\tan(0.40)} = \frac{0.55}{0.4228} = 1.301 \text{ m}$$

Step 2: Individual steer angles.

$$\begin{aligned} \delta_{\text{left (inner)}} &= \arctan\left(\frac{0.55}{1.301 - 0.18}\right) = \arctan\left(\frac{0.55}{1.121}\right) = \arctan(0.4907) = 0.456 \text{ rad} = 26.1^\circ \\ \delta_{\text{right (outer)}} &= \arctan\left(\frac{0.55}{1.301 + 0.18}\right) = \arctan\left(\frac{0.55}{1.481}\right) = \arctan(0.3714) = 0.355 \text{ rad} = 20.3^\circ \end{aligned}$$

Comparison: Centre steer = 22.9° , inner = 26.1° , outer = 20.3° . The inner wheel steers 3.2° more, and the outer 2.6° less, than the centre equivalent.

4.5 AuroraMR Code

Listing 4.1: Ackermann simulation in AuroraMR

```
1 s = amr.MotionSession.create(  
2     amr.pose(0, 0, 0),  
3     amr.KinematicsModel.ACKERMANN,  
4     dt=0.02,  
5     ackermann=amr.AckermannParams(  
6         wheelbase=0.55,  
7         track_width=0.36,  
8         max_steering_angle=0.48, # ~27.5 deg  
9         max_speed=1.0  
10    )  
11 )  
12  
13 s.forward(1.5, speed=0.6) # straight at 0.6 m/s  
14 s.turn_left(math.radians(40), 0.8) # arc left ~40 deg  
15 s.forward(0.8, speed=0.5) # straight
```

Chapter 5

Mecanum (Holonomic) Kinematics

5.1 Model Description

A **mecanum wheel** has small passive rollers mounted at 45° to the wheel axle. This allows the robot to move in any direction without rotating — a property called **holonomic motion**. The robot has:

- Four wheels: Front-Left (FL), Front-Right (FR), Rear-Left (RL), Rear-Right (RR)
- $l_y = \text{half_length_y}$: distance from centre to front/rear axle [m]
- $l_x = \text{half_width_x}$: distance from centre to left/right wheels [m]
- Wheel linear speeds $v_{FL}, v_{FR}, v_{RL}, v_{RR}$ [m/s at ground contact]

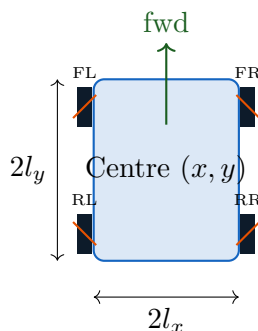


Figure 5.1: X-configuration mecanum robot. Orange lines show 45° roller axes. l_x and l_y are half-widths from centre.

5.2 Inverse Kinematics: Wheel Speeds to Body Twist

For the X-configuration mecanum wheel arrangement, the body-frame twist (v_x, v_y, ω) — right, forward, CCW — is:

Key Equation: Mecanum Forward Kinematics (Wheel Speeds $\rightarrow$ Body Twist)

$$\begin{aligned} v_y &= \frac{1}{4}(v_{FL} + v_{FR} + v_{RL} + v_{RR}) \\ v_x &= \frac{1}{4}(-v_{FL} + v_{FR} + v_{RL} - v_{RR}) \\ \omega &= \frac{-v_{FL} + v_{FR} - v_{RL} + v_{RR}}{4(l_y + l_x)} \end{aligned}$$

Derivation

Each mecanum wheel contributes a velocity component to the robot. For the X-configuration, the 4×3 Jacobian relating wheel speeds to body twist is:

$$\begin{pmatrix} v_y \\ v_x \\ \omega \end{pmatrix} = \frac{1}{4} \begin{pmatrix} 1 & 1 & 1 & 1 \\ -1 & 1 & 1 & -1 \\ \frac{-1}{l_y+l_x} & \frac{1}{l_y+l_x} & \frac{-1}{l_y+l_x} & \frac{1}{l_y+l_x} \end{pmatrix} \begin{pmatrix} v_{FL} \\ v_{FR} \\ v_{RL} \\ v_{RR} \end{pmatrix}$$

The sign pattern for each row encodes the geometric contribution of each wheel:

- **Forward** (v_y): All four wheels contribute equally; sign = +1 for all.
- **Lateral** (v_x): FL and RR rollers oppose lateral motion; FR and RL assist it. Sign pattern: $(-1, +1, +1, -1)$.
- **Rotation** (ω): The denominator $4(l_y + l_x)$ is the sum of all wheel lever arms. Sign pattern: $(-1, +1, -1, +1)$.

5.3 World Frame Integration

Once the body-frame twist (v_x, v_y, ω) is computed, we rotate it into the world frame using $\mathbf{R}(\theta)$:

Key Equation: Mecanum World-Frame Velocity

$$\begin{aligned} \dot{x}_w &= v_y(-\sin \theta) + v_x \cos \theta \\ \dot{y}_w &= v_y \cos \theta + v_x \sin \theta \end{aligned}$$

Derivation

The body twist is a vector in the body frame: $\mathbf{v}_b = (v_x, v_y)^\top$. Rotating to world frame:

$$\begin{pmatrix} \dot{x}_w \\ \dot{y}_w \end{pmatrix} = \mathbf{R}(\theta) \begin{pmatrix} v_x \\ v_y \end{pmatrix} = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix} \begin{pmatrix} v_x \\ v_y \end{pmatrix} = \begin{pmatrix} v_x \cos \theta - v_y \sin \theta \\ v_x \sin \theta + v_y \cos \theta \end{pmatrix}$$

Rearranging:

$$\dot{x}_w = v_y(-\sin \theta) + v_x \cos \theta, \quad \dot{y}_w = v_y \cos \theta + v_x \sin \theta$$

The Euler integration step is then:

$$\begin{aligned}x' &= x + \dot{x}_w \Delta t \\y' &= y + \dot{y}_w \Delta t \\\theta' &= \text{wrap}_\pi(\theta + \omega \Delta t)\end{aligned}$$

5.4 Inverse Kinematics: Desired Twist $\rightarrow$ Wheel Speeds

For the inverse problem (planning), given desired (v_y, v_x, ω) :

Key Equation: Mecanum Inverse Kinematics (Body Twist $\rightarrow$ Wheel Speeds)

$$\begin{aligned}v_{FL} &= v_y - v_x - (l_y + l_x) \omega \\v_{FR} &= v_y + v_x + (l_y + l_x) \omega \\v_{RL} &= v_y + v_x - (l_y + l_x) \omega \\v_{RR} &= v_y - v_x + (l_y + l_x) \omega\end{aligned}$$

These follow directly from inverting the 4×3 Jacobian (pseudo-inverse for the over-determined case).

5.5 Special Motion Patterns

Motion	v_{FL}	v_{FR}	v_{RL}	v_{RR}
Forward (v)	$+v$	$+v$	$+v$	$+v$
Backward (v)	$-v$	$-v$	$-v$	$-v$
Strafe right (v)	$-v$	$+v$	$+v$	$-v$
Strafe left (v)	$+v$	$-v$	$-v$	$+v$
Rotate CCW (a)	$-a$	$+a$	$-a$	$+a$
Rotate CW (a)	$+a$	$-a$	$+a$	$-a$

where $a = \omega(l_y + l_x)$ for in-place rotation.

5.6 Worked Numerical Examples

Worked Example: Mecanum Forward Motion

Setup: $l_y = 0.26$ m, $l_x = 0.20$ m. Wheel speeds: all $v_i = 0.5$ m/s.

Step 1: Body twist.

$$\begin{aligned}v_y &= \frac{1}{4}(0.5 + 0.5 + 0.5 + 0.5) = \frac{1}{4}(2.0) = 0.500 \text{ m/s} \\v_x &= \frac{1}{4}(-0.5 + 0.5 + 0.5 - 0.5) = \frac{1}{4}(0) = 0.000 \text{ m/s} \\\omega &= \frac{-0.5 + 0.5 - 0.5 + 0.5}{4(0.26 + 0.20)} = \frac{0}{1.84} = 0.000 \text{ rad/s}\end{aligned}$$

Pure forward motion. ✓

Step 2: World frame at $\theta = 0$.

$$\dot{x}_w = (0.5)(-\sin 0) + (0) \cos 0 = 0 \text{ m/s}$$

$$\dot{y}_w = (0.5) \cos 0 + (0) \sin 0 = 0.5 \text{ m/s}$$

Robot moves north at 0.5 m/s. ✓

Worked Example: Mecanum Strafe Right

Setup: Same robot. Wheel speeds: $v_{FL} = -0.6$, $v_{FR} = +0.6$, $v_{RL} = +0.6$, $v_{RR} = -0.6$ m/s.

Step 1: Body twist.

$$v_y = \frac{1}{4}(-0.6 + 0.6 + 0.6 - 0.6) = 0.000 \text{ m/s}$$

$$v_x = \frac{1}{4}(0.6 + 0.6 + 0.6 + 0.6) = \frac{2.4}{4} = 0.600 \text{ m/s}$$

$$\omega = \frac{0.6 + 0.6 - 0.6 + (-0.6)}{4(0.46)} = \frac{0.6 + 0.6 - 0.6 - 0.6}{1.84} = \frac{0}{1.84} = 0 \text{ rad/s}$$

Pure lateral motion, no rotation. ✓

Step 2: World frame at $\theta = 0$.

$$\dot{x}_w = (0)(-\sin 0) + (0.6)(1) = 0.600 \text{ m/s (East)}$$

$$\dot{y}_w = (0)(1) + (0.6)(0) = 0.000 \text{ m/s}$$

Robot strafes east at 0.6 m/s without rotating. ✓

Worked Example: Mecanum In-Place Rotation

Setup: $l_y = 0.26$ m, $l_x = 0.20$ m. Desired $\omega = 1.0$ rad/s CCW.

Step 1: Compute a .

$$a = \omega(l_y + l_x) = 1.0 \times (0.26 + 0.20) = 0.460 \text{ m/s}$$

Step 2: Wheel speeds.

$$v_{FL} = v_{RL} = -0.460 \text{ m/s}, \quad v_{FR} = v_{RR} = +0.460 \text{ m/s}$$

Verification:

$$v_y = \frac{1}{4}(-0.46 + 0.46 + (-0.46) + 0.46) = 0 \checkmark$$

$$v_x = \frac{1}{4}(0.46 + 0.46 + (-0.46) + (-0.46)) = 0 \checkmark$$

$$\omega = \frac{0.46 + 0.46 - (-0.46) + 0.46}{4(0.46)} = \frac{4 \times 0.46}{4 \times 0.46} = 1.0 \text{ rad/s} \checkmark$$

Worked Example: Mecanum — Combined Motion (Forward + Strafe + Rotate)

Setup: Desired $v_y = 0.4$, $v_x = 0.2$, $\omega = 0.5$ rad/s. $l_y = 0.26$, $l_x = 0.20$.

Step 1: Compute $k = l_y + l_x = 0.46$ m.

Step 2: Apply inverse kinematics.

$$v_{FL} = 0.4 - 0.2 - 0.46 \times 0.5 = 0.2 - 0.23 = -0.030 \text{ m/s}$$

$$v_{FR} = 0.4 + 0.2 + 0.23 = 0.830 \text{ m/s}$$

$$v_{RL} = 0.4 + 0.2 - 0.23 = 0.370 \text{ m/s}$$

$$v_{RR} = 0.4 - 0.2 + 0.23 = 0.430 \text{ m/s}$$

Verification (forward kinematics round-trip):

$$v_y = \frac{1}{4}(-0.030 + 0.830 + 0.370 + 0.430) = \frac{1.600}{4} = 0.400 \text{ m/s} \checkmark$$

$$v_x = \frac{1}{4}(0.030 + 0.830 + 0.370 - 0.430) = \frac{0.800}{4} = 0.200 \text{ m/s} \checkmark$$

$$\omega = \frac{0.030 + 0.830 - 0.370 + 0.430}{4(0.46)} = \frac{0.920}{1.84} = 0.500 \text{ rad/s} \checkmark$$

5.7 AuroraMR Code

Listing 5.1: Mecanum simulation in AuroraMR

```

1  s = amr.MotionSession.create(
2      amr.pose(0, 0, 0),
3      amr.KinematicsModel.MECANUM,
4      dt=0.015,
5      mecanum=amr.MecanumParams(
6          half_length_y=0.26,
7          half_width_x=0.20,
8          max_wheel_speed=2.5
9      )
10 )
11
12 s.forward(0.7, speed=0.6)           # pure forward
13 s.strafe_right(0.5, speed=0.55)     # pure lateral
14 s.turn_left(math.pi/4, 1.0)        # in-place CCW
15
16 # Low-level: direct wheel speeds for 0.8 s (FL, FR, RL, RR)
17 s.mecanum_drive_wheels(0.4, 0.4, 0.5, 0.5, duration=0.8)

```

Chapter 6

Wheel Contact Point Geometry

6.1 Why Track Contact Points?

Recording where each wheel physically touches the ground is important for: visualising tyre paths, checking for obstacle collisions, and validating kinematic consistency. All contact-point transforms use the rotation matrix $\mathbf{R}(\theta)$.

6.2 Two-Wheel / Differential Contacts

Left and right contacts are offset $\pm W/2$ from the robot centre along the body x -axis (perpendicular to forward direction):

$$\mathbf{b}_L = \begin{pmatrix} -W/2 \\ 0 \end{pmatrix}, \quad \mathbf{b}_R = \begin{pmatrix} +W/2 \\ 0 \end{pmatrix}$$

World positions:

$$\mathbf{c}_{L,R} = \mathbf{R}(\theta) \mathbf{b}_{L,R} + \begin{pmatrix} x \\ y \end{pmatrix}$$

Expanding:

$$c_{L,x} = x + (-W/2) \cos \theta - 0 \cdot \sin \theta = x - \frac{W}{2} \cos \theta$$

$$c_{L,y} = y + (-W/2) \sin \theta + 0 \cdot \cos \theta = y - \frac{W}{2} \sin \theta$$

$$c_{R,x} = x + \frac{W}{2} \cos \theta, \quad c_{R,y} = y + \frac{W}{2} \sin \theta$$

Worked Example: Tyre Contacts

Robot at $(1.0, 1.0, \pi/4)$ (NW facing), $W = 0.4$ m.

$$\mathbf{R}(\pi/4) = \begin{pmatrix} 0.707 & -0.707 \\ 0.707 & 0.707 \end{pmatrix}$$

$$\mathbf{c}_L = \begin{pmatrix} 0.707 & -0.707 \\ 0.707 & 0.707 \end{pmatrix} \begin{pmatrix} -0.2 \\ 0 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} -0.141 \\ -0.141 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 0.859 \\ 0.859 \end{pmatrix}$$

$$\mathbf{c}_R = \begin{pmatrix} 0.707 & -0.707 \\ 0.707 & 0.707 \end{pmatrix} \begin{pmatrix} 0.2 \\ 0 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 0.141 \\ 0.141 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 1.141 \\ 1.141 \end{pmatrix}$$

6.3 Four-Wheel (Ackermann) Contacts

The four corners in the body frame, with the rear axle midpoint as origin:

$$\text{RL} = \begin{pmatrix} -W/2 \\ 0 \end{pmatrix}, \quad \text{RR} = \begin{pmatrix} +W/2 \\ 0 \end{pmatrix}, \quad \text{FL} = \begin{pmatrix} -W/2 \\ L \end{pmatrix}, \quad \text{FR} = \begin{pmatrix} +W/2 \\ L \end{pmatrix}$$

Each transforms as $\mathbf{c}_i = \mathbf{R}(\theta) \mathbf{b}_i + (x, y)^\top$.

6.4 Mecanum Four-Wheel Contacts

Body-frame positions relative to the geometric centre:

$$\text{FL} = \begin{pmatrix} -l_x \\ l_y \end{pmatrix}, \quad \text{FR} = \begin{pmatrix} +l_x \\ l_y \end{pmatrix}, \quad \text{RL} = \begin{pmatrix} -l_x \\ -l_y \end{pmatrix}, \quad \text{RR} = \begin{pmatrix} +l_x \\ -l_y \end{pmatrix}$$

Chapter 7

Numerical Accuracy and Integration Error

7.1 Euler Method Error Analysis

The Euler integration used in AuroraMR is first-order. The *local truncation error* per step is $\mathcal{O}(\Delta t^2)$ and the *global error* over a fixed time interval T is $\mathcal{O}(\Delta t)$.

For the heading integration:

$$\theta(t + \Delta t) = \theta(t) + \omega \Delta t + \underbrace{\frac{1}{2} \dot{\omega} \Delta t^2}_{\text{Euler error}} + \mathcal{O}(\Delta t^3)$$

For constant ω , the heading error after N steps ($T = N\Delta t$) is zero (exact). The position error arises because the heading used at each step is the *start-of-step* heading, not the midpoint:

$$\text{Position error} \approx \frac{v \omega T^2}{2} \Delta t$$

Worked Example: Euler Heading Error for a Circle

Robot drives a circle: $v = 1.0$ m/s, $\omega = 1.0$ rad/s (period $T = 2\pi$ s). After one revolution with $\Delta t = 0.02$ s ($N = 314$ steps):

$$\text{Final position error} \approx \frac{v \omega (2\pi)^2}{2} \Delta t = \frac{1.0 \times 1.0 \times 39.5}{2} \times 0.02 \approx 0.39 \text{ cm}$$

For a visualisation library this is acceptable; for hardware deployment, use RK4.

7.2 Runge-Kutta 4 (RK4) Alternative

The fourth-order RK4 method has global error $\mathcal{O}(\Delta t^4)$. For the unicycle ODE $\dot{\mathbf{q}} = f(\mathbf{q}, u)$:

$$k_1 = f(\mathbf{q}_n, u), \quad k_2 = f\left(\mathbf{q}_n + \frac{\Delta t}{2} k_1, u\right), \quad k_3 = f\left(\mathbf{q}_n + \frac{\Delta t}{2} k_2, u\right), \quad k_4 = f(\mathbf{q}_n + \Delta t k_3, u)$$

$$\mathbf{q}_{n+1} = \mathbf{q}_n + \frac{\Delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4)$$

For constant (v, ω) inputs (as in AuroraMR’s segment-based motion), Euler and RK4 produce similar results. RK4 is significantly better when ω is rapidly changing.

Chapter 8

Summary of All Equations

Formula	Equation	Status
Angle wrap	$\text{wrap}_\pi(\theta) = ((\theta + \pi) \bmod 2\pi) - \pi$	✓
Rotation matrix	$\mathbf{R}(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$	✓
Forward direction	$\mathbf{f} = (-\sin \theta, \cos \theta)^\top$	✓
Unicycle x -update	$x' = x - v \sin(\theta) \Delta t$	✓
Unicycle y -update	$y' = y + v \cos(\theta) \Delta t$	✓
Unicycle θ -update	$\theta' = \text{wrap}_\pi(\theta + \omega \Delta t)$	✓
Diff. drive v	$v = (v_R + v_L)/2$	✓
Diff. drive ω	$\omega = (v_R - v_L)/W$	✓
Diff. in-place	$v_L = -\omega W/2, v_R = +\omega W/2$	✓
Bicycle ω	$\omega = v \tan(\delta)/L$	✓
Ackermann R	$R = L/\tan(\delta_c)$	✓
Ackermann δ_{inner}	$\arctan(L/(R - W/2))$	✓
Ackermann δ_{outer}	$\arctan(L/(R + W/2))$	✓
Mecanum v_y	$(v_{FL} + v_{FR} + v_{RL} + v_{RR})/4$	✓
Mecanum v_x	$(-v_{FL} + v_{FR} + v_{RL} - v_{RR})/4$	✓
Mecanum ω	$(-v_{FL} + v_{FR} - v_{RL} + v_{RR})/(4(l_y + l_x))$	✓
Mecanum world vel.	$\dot{x}_w = v_y(-\sin \theta) + v_x \cos \theta$	✓
Mecanum inv. FL	$v_{FL} = v_y - v_x - (l_y + l_x)\omega$	✓
Contact transform	$\mathbf{c} = \mathbf{R}(\theta) \mathbf{b} + (x, y)^\top$	✓
Heading to point	$h = \text{atan2}(-\Delta x, \Delta y)$	✓
Playback speed	$\Delta t_{\text{eff}} = \max(1, \Delta t_{\text{base}}/s)$	✓

Chapter 9

Practice Problems

- P1.** A unicycle robot starts at $(0,0,0)$. It drives with $v = 0.5$ m/s and $\omega = 0$ for $t = 4$ s using $\Delta t = 1$ s. Compute the final pose analytically and verify with Euler integration.
- P2.** A differential drive robot has $W = 0.50$ m. Wheel speeds are $v_L = 0.2$ m/s and $v_R = 0.8$ m/s. (a) Compute v and ω . (b) Compute the turning radius R . (c) Which way does the robot turn?
- P3.** An Ackermann vehicle has $L = 0.60$ m, $W = 0.40$ m. The driver applies a centre steer angle of $\delta_c = 0.30$ rad. (a) Compute the turning radius R . (b) Compute the exact left and right wheel steer angles. (c) How much do they differ from δ_c ?
- P4.** A mecanum robot ($l_y = 0.25$ m, $l_x = 0.20$ m) has wheel speeds $v_{FL} = 0.6$, $v_{FR} = 0.4$, $v_{RL} = 0.4$, $v_{RR} = 0.6$ m/s. Compute v_y , v_x , and ω . Describe the resulting motion.
- P5. [Design]** You want a mecanum robot to move diagonally: forward at 0.5 m/s and right at 0.5 m/s simultaneously, with no rotation. Compute the required wheel speeds using the inverse kinematics equations. Verify by substituting back into the forward kinematics.
- P6.** A robot has heading $\theta = 250^\circ$. There is a target at relative position $(\Delta x, \Delta y) = (+1.0, +0.5)$. (a) Compute the required heading $h = \text{atan2}(-\Delta x, \Delta y)$ in degrees. (b) Compute the angular error $\Delta\theta = \text{wrap}_\pi(h - \theta)$. (c) Should the robot turn left (CCW) or right (CW)?
- P7.** A unicycle robot completes a circle of radius $R = 1.0$ m at $v = 1.0$ m/s. (a) What is the required ω ? (b) What is the circumference? (c) How many Euler steps at $\Delta t = 0.02$ s are needed for one revolution? (d) Estimate the position error using the formula from Chapter 6.

Appendix A

Quick Reference: AuroraMR Parameter Defaults

Class	Parameter	Default	Unit
UnicycleParams	track_width	0.4	m
	max_linear_speed	1.0	m/s
	max_angular_speed	2.0	rad/s
DifferentialParams	track_width	0.4	m
	wheel_radius	0.1	m
	max_wheel_speed	2.0	m/s
AckermannParams	wheelbase	0.55	m
	track_width	0.35	m
	max_steering_angle	0.55	rad
	max_speed	1.2	m/s
MecanumParams	half_length_y	0.25	m
	half_width_x	0.20	m
	max_wheel_speed	2.0	m/s

Coordinate Convention Reminder

- $\theta = 0$: North ($+y$) $\theta = \pi/2$: West ($-x$)
- $\theta = \pi$: South ($-y$) $\theta = -\pi/2$: East ($+x$)
- Positive ω : CCW (left turn when viewed from above)
- Forward direction in world: $(-\sin \theta, \cos \theta)$